The PSPR Planning Algorithm

Constraint Partitioning, Relaxed Penalty Merging, and Hierarchical Refinement

PSPR project

October 22, 2026

1 The planning problem

A grounded hierarchical planning problem is a tuple

$$P = \langle F,\, A,\, C,\, M,\, s_0,\, TN_0,\, g \rangle, \tag{1}$$

where F is a finite set of ground atoms, A a set of primitive action schemas, C a set of compound task schemas, M a set of methods, $s_0 \subseteq F$ the initial state, $TN_0 = \langle T_0, \prec_0, \approx_0 \rangle$ the initial task network, and g a state-goal formula, possibly absent. For a primitive action a with precondition $\mathrm{pre}_a \subseteq F$, add effects $\mathrm{add}_a \subseteq F$, and delete effects $\mathrm{del}_a \subseteq F$, the state transition is

$$\tau_a(s) = \begin{cases} (s \setminus \mathrm{del}_a) \cup \mathrm{add}_a & \text{if } s \models \mathrm{pre}_a, \\ \text{undefined} & \text{otherwise.} \end{cases} \tag{2}$$

A task network is $TN = \langle T, \prec, \approx \rangle$: a finite multiset of task occurrences, an acyclic ordering $\prec \subseteq T \times T$, and equality and inequality constraints $\approx$. A method

$$m = \langle c(\vec{x}),\, \phi_m,\, TN_m \rangle \tag{3}$$

decomposes the compound task $c(\vec{x})$ into the subtask network TN_m. A decomposition step replaces an occurrence $t = c(\vec{t})$ by a grounded method instance $m\sigma$ and updates the ordering by

$$\prec' = \prec \;\cup\; \prec_m \sigma \;\cup\; \{(u, v) \mid u \prec t,\; v \in T_m\sigma\} \;\cup\; \{(v, w) \mid v \in T_m\sigma,\; t \prec w\}. \tag{4}$$

A decomposition tree D is primitive when every task in its frontier $L(D)$ is a primitive action. A plan $\pi = \langle a_1, \ldots, a_k \rangle$ is a linearisation of $L(D)$ that respects $\prec_D$ and $\approx_D$; it is executable from s_0 when each a_i is applicable in the state produced by its predecessors, and it is a solution when, in addition, the final state satisfies g.

Notation. For a task occurrence t we write $\mathrm{head}(t)$ for its task symbol and $\mathrm{args}(t)$ for its ground arguments. The projection of a set of ground atoms onto a subset $X \subseteq F$ is $s[X] = \{a \in s \mid a \in X\}$.

2 The abstraction computed by the evaluation tree

The evaluation tree does not solve P directly. It solves a *top-level projection* $\alpha(P)$ in which compound tasks are opaque. For every grounded task occurrence $t \in T_0$ introduce a marker atom $\mathrm{exec}_t \notin F$. A primitive occurrence t with $\mathrm{head}(t) = a$ is compiled into the operator

$$\mathrm{pre}(o_t) = \mathrm{pre}_a \;\wedge \bigwedge_{u \prec_0 t} \mathrm{exec}_u, \qquad \mathrm{add}(o_t) = \mathrm{add}_a \cup \{\mathrm{exec}_t\}, \qquad \mathrm{del}(o_t) = \mathrm{del}_a, \tag{5}$$

and a compound occurrence t with method instances $M(t)$ gets one operator per instance $m\sigma$:

$$\mathrm{pre}(o_{t,m}) = \phi_m\sigma \ \wedge \bigwedge_{u \prec_0 t} \mathrm{exec}_u, \qquad \mathrm{add}(o_{t,m}) = \{\mathrm{exec}_t\}, \qquad \mathrm{del}(o_{t,m}) = \varnothing. \tag{6}$$

The ordering of TN_0 is thus enforced operationally by predecessor markers, not by a separate constraint layer. The state goal g is not part of the abstraction; it is checked after planning, either by the strict validator or by the refinement stage of Section 8.

Remark 2.1 (The liftability gap)**.** *The projection is neither a sound relaxation nor a complete abstraction of P. It can* reject *a valid HTN plan: if the subtasks of a method for t_1 establish an atom p that is a precondition of a method for a later task t_2, then in $\alpha(P)$ the operator for t_2's method cannot fire, because t_1 only writes exec_{t_1}. Conversely, it can* accept *a top-level plan that no decomposition supports: an abstract method operator only tests ϕ_m, while the first subtask of $m\sigma$ may be inapplicable in s_0. No inclusion holds in general between the top-level plans of $\alpha(P)$ and the top-level plans induced by HTN solutions of P. This gap is why PSPR offers the bounded refinement of Section 8; the default path returns a plan of the projection plus an independent validation report.*

3 Grounding and the task/constraint incidence graph

3.1 Grounding

The free parameters of TN_0 are enumerated over the typed constants and objects of the problem. A parameter ranges over the symbols whose declared type is a subtype of the parameter type. If the enumeration produces several groundings, their task lists are concatenated and each grounding contributes its own copy of the ordering and equality constraints. The number of assignments is bounded by the grounding budget B_g (`EvaluationOptions::max_groundings`, default 256 in the CLI).

Action and method formulas are compiled in four steps:

1. quantifiers $\exists$ and $\forall$ are expanded over the typed universe;
2. *Imply*, *Xor* and composite *Not* are rewritten into negation normal form over literals;
3. *Equals* is evaluated against the current bindings;
4. preconditions are converted to disjunctive normal form (DNF), and effects are compiled into conditional effects. Every DNF clause and every disjunctive effect alternative becomes one operator variant.

The DNF expansion is capped at 4096 clauses or alternatives per formula. *Probabilistic* is the only constructor that cannot be represented; it is counted in `EvaluationTree::unmodeled` and makes leaf construction fail.

3.2 The incidence graph

Let $\mathcal{T} = \{t_1, \ldots, t_{n_1}\}$ be the grounded task occurrences and let $\mathcal{K} = \{c_1, \ldots, c_{n_2}\}$ be the constraints they touch. A constraint node is one of:

- an atom occurring in a matching action schema (precondition or effect) or in a matching method precondition (and, when summaries are enabled, in a summary effect);
- an ordering pair (t_i, t_j) of the network;
- an equality or inequality constraint between two ground terms.

The incidence graph is the bipartite graph with adjacency matrix

$$A_{ij} = \begin{cases} 1 & \text{if task } t_i \text{ touches constraint } c_j, \\ 0 & \text{otherwise.} \end{cases} \tag{7}$$

Its connected components are used as *required groupings*: independent parts of the network are clustered separately and never mixed by the spectral step. Explicit groupings can be supplied through `EvaluationOptions::groupings`; tasks not mentioned in any grouping become singleton groups.

4 Spectral clustering of tasks

Each grouping is clustered independently on its induced subgraph. Let the group have n_1 type-I nodes, n_2 type-II nodes, m edges and density

$$p = \frac{m}{n_1 n_2}. \tag{8}$$

Two regimes are distinguished by the signal condition $n_1 n_2 p^2 \geq \log n_1$.

4.1 Dense regime: Spec and AdaSpec

Let

$$S = H(AA^\top) \tag{9}$$

be the *hollowed* Gram matrix: $S_{ij} = (AA^\top)_{ij}$ for $i \neq j$ and $S_{ii} = 0$. Hollowing is essential because the diagonal of $AA^\top$ is of order $n_2 p_{\max}$, while the population diagonal of $PP^\top$ is of order $n_2 p_{\max}^2$; keeping it would dominate the spectrum. Let $S = U\Lambda U^\top$ with $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{n_1}$ and orthonormal columns $u_1, \ldots, u_{n_1}$.

AdaSpec selects the embedding rank

$$r = \max\left\{i \in \{1, \ldots, n_1 - 1\} : \ \lambda_i - \lambda_{i+1} > \theta\right\}, \qquad \theta = \frac{n_1 n_2 p^2}{\log\log n_1}, \tag{10}$$

where the threshold is replaced by $n_1 n_2 p^2$ when $\log\log n_1 \leq 0$ or is non-finite. If no gap exceeds θ, the implementation falls back to the largest gap, provided it exceeds $10^{-9}\max(|\lambda_1|, 1)$; otherwise the group stays a single cluster. The embedding of task i is

$$u(i) = \big(U_{i,1}, \ldots, U_{i,r}\big) \in \mathbb{R}^r, \tag{11}$$

and the number of communities is $k = r$.

4.2 Sparse regime: normalized Laplacian

When $n_1 n_2 p^2 < \log n_1$ the hollowed Gram matrix is too noisy. The implementation uses the symmetric normalized Laplacian of the similarity graph S:

$$L = I - D^{-1/2} S D^{-1/2}, \qquad D_{ii} = \sum_j S_{ij}, \tag{12}$$

with spectrum $0 = \mu_1 \leq \mu_2 \leq \cdots \leq \mu_{n_1}$. The number of communities is the classic eigengap estimate

$$k = \arg\max_{c \geq 1} \big(\mu_{c+1} - \mu_c\big), \tag{13}$$

and the embedding is given by the eigenvectors of $\mu_2, \ldots, \mu_k$.

4.3 Assignment and clusters

Both regimes partition the embedded rows with k-medians:

$$\min_{\mu_1,\dots,\mu_k} \sum_{i=1}^{n_1} \big\| u(i) - \mu_{\ell(i)} \big\|_2, \tag{14}$$

where rows are assigned to the nearest centre in the Euclidean norm and each centre is updated to the coordinate-wise median of its cell. The iteration is deterministic, seeded with SplitMix64. Cluster j consists of its type-I tasks $\mathcal{T}_j$ together with every type-II node adjacent to at least one of them:

$$\mathcal{K}_j = \big\{ c \in \mathcal{K} : \exists t \in \mathcal{T}_j,\ A_{t,c} = 1 \big\}. \tag{15}$$

A constraint is *shared* when it belongs to more than one $\mathcal{K}_j$:

$$\mathcal{S} = \Big\{ c \in \mathcal{K} : \big| \{ j : c \in \mathcal{K}_j \} \big| > 1 \Big\}, \tag{16}$$

and *local* to cluster j otherwise. Local constraints are solved inside the leaf; shared constraints are delegated to the merge nodes.

5 Agglomeration and the evaluation tree

5.1 Pairwise distance

For two clusters j and j' the implementation uses the Jaccard distance

$$d(j, j') = 1 - \frac{|\mathcal{K}_j \cap \mathcal{K}_{j'}|}{|\mathcal{K}_j \cup \mathcal{K}_{j'}|}, \tag{17}$$

with $d(j, j') = 0$ when both constraint sets are empty.

5.2 The merge-level ILP

Let k be the number of clusters and L the number of levels ($L = k - 1$ by default). The hierarchy is represented by integer merge-level variables

$$M_{ab} \in \{1, \dots, L\}, \qquad a \neq b, \tag{18}$$

where M_{ab} is the level at which a and b first belong to the same cluster. For every ordered triple (a, b, c) of distinct clusters the formulation introduces binary indicators O_{abc}, Z^{ab}_{abc} and Z^{bc}_{abc}, and the constraints

$$-L \ \le M_{ac} - M_{ab} - (L+1) O_{abc} \ \le\ 0, \tag{19}$$

$$-L \ \le M_{ab} - M_{ac} - (L+1) Z^{ab}_{abc} + 1 \ \le\ 0, \tag{20}$$

$$-L \ \le M_{bc} - M_{ac} - (L+1) Z^{bc}_{abc} + 1 \ \le\ 0, \tag{21}$$

$$Z^{ab}_{abc} + Z^{bc}_{abc} \ \ge\ 1. \tag{22}$$

Constraint (19) forces $O_{abc} = 1$ exactly when $M_{ac} < M_{ab}$, i.e. when b joins a before c does. Constraints (20)–(22) encode the dendrogram validity condition that M_{ac} cannot be strictly larger than both M_{ab} and M_{bc}; equivalently, none of the three points may join the other two last. The clustering objective is

$$Q(A) = \sum_{a} \sum_{b \neq a} \sum_{c \neq a,b} \big(d(a, c) - d(a, b) \big)\, O_{abc}, \tag{23}$$

which is the sum, over triples where a and b merge before a and c, of the gain obtained by placing the closer neighbour b first.

5.3 Constraint locality

For every shared constraint c with cluster set $S_c = \{j : c \in \mathcal{K}_j\}$ the formulation adds a continuous height variable $h_c \in [1, L]$ and the constraints

$$h_c \;\geq\; M_{ab} \qquad \text{for all distinct } a, b \in S_c. \tag{24}$$

Since the objective penalizes $\sum_c h_c$, at optimality

$$h_c = \max_{a,b \in S_c} M_{ab}, \tag{25}$$

which is exactly the level of the lowest common ancestor of the clusters that share c. The complete hierarchy objective is therefore

$$\max_{A} \;\; Q(A) - \gamma\, G(A), \qquad G(A) = \sum_{c \in \mathcal{S}} h_c, \tag{26}$$

where γ is `Options::global_weight` (default 1). A constraint handled by the first merge costs 1; one pushed to the root costs L. The penalty keeps shared constraints low in the tree, where they are checked against smaller subplans.

5.4 From merge levels to a binary dendrogram

At every level ℓ the clusters are the connected components of the relation "$M_{ab} \leq \ell$", computed with a union-find structure. A rooted binary dendrogram is built by walking the levels from 1 to L: each non-singleton cluster at level ℓ is formed from the clusters of level $\ell - 1$ that it contains, and a multi-way merge is binarised by folding its children in ascending order. Leaves are the k clusters; the $k - 1$ internal nodes are created in chronological order, and internal node $k + \ell$ receives evaluation id ℓ.

Each shared constraint is attached to the *first* internal node, in creation order, whose member set contains every cluster in S_c. Since nodes are created bottom-up, this is the lowest common ancestor of S_c. The merge node therefore receives exactly the constraints whose clusters first meet there.

5.5 The scheduling ILP

Let I be the set of internal nodes, $N = |I| = k - 1$, and let E_A be the parent–child edges of the dendrogram, directed from child to parent. Binary variables $x_{v,t}$ indicate that internal node v is evaluated at time t:

$$\sum_{t=1}^{N} x_{v,t} = 1 \qquad \forall v \in I, \tag{27}$$

$$\sum_{v \in I} x_{v,t} = 1 \qquad \forall t \in \{1, \ldots, N\}, \tag{28}$$

with evaluation position $p_v = \sum_{t=1}^{N} t\, x_{v,t}$. Every child is evaluated before its parent, and an optional evaluation DAG $B = (V_B, E_B)$ adds precedences:

$$p_u + 1 \leq p_v \qquad \forall (u, v) \in E_A,\; u, v \in I, \tag{29}$$

$$p_{f(a)} + 1 \leq p_{f(b)} \qquad \forall (a, b) \in E_B, \tag{30}$$

where f maps DAG vertices to internal nodes. The ILP minimizes

$$C(A, \sigma) = \sum_{v \in I} p_v. \tag{31}$$

The union of (29) and (30) must be acyclic; otherwise the schedule is rejected. The resulting order σ is a topological order of the merge nodes.

5.6 Joint objective

The selected tree and schedule are scored by

$$Q(A) - \lambda\, C(A, \sigma) - \gamma\, G(A), \tag{32}$$

where λ is `Options::lambda` (default 0). Since every complete binary hierarchy over k leaves has exactly $k-1$ internal nodes, C is nearly constant across hierarchies and the ILP of (26) dominates the choice. An optional planning-style search over partial dendrograms can be enabled (`use_search`); it reuses the generic best-first engine and picks its candidate only when it improves (32). In the default configuration only the exact ILP is solved.

6 Leaf subproblems

For cluster j let L_j be the set of atoms in its local constraints. The leaf planning task has binary variables

$$V_j = \{x_a : a \in L_j\} \;\cup\; \{e_t : t \in \mathcal{T}_j\}, \tag{33}$$

where x_a is the truth value of atom a and e_t is the execution marker of task t. The initial assignment is

$$x_a = 1 \iff a \in s_0, \qquad e_t = 0, \tag{34}$$

and the goal is

$$G_j = \{e_t = 1 : t \in \mathcal{T}_j\}. \tag{35}$$

Shared atoms are dropped from the leaf entirely: they neither appear as preconditions nor as effects of leaf operators. This is what makes the leaves independent; the missing shared atoms are the interface that the merge must repair.

6.1 Ordering closure

Let $\prec_j$ be the ordering edges induced by the local ordering constraints between tasks of $\mathcal{T}_j$. The implementation computes the transitive closure $\prec_j^*$ (Floyd–Warshall over the cluster positions) and rejects a cyclic closure. Every operator for task t contains the predecessor markers

$$\prec_j^*(t) = \{e_u = 1 : u \prec_j^* t\}. \tag{36}$$

6.2 Action variants

Let ϕ be a precondition in DNF, $\phi = \bigvee_{\kappa \in K} \bigwedge_{\ell \in \kappa} \ell$, and let $\alpha = \{(\gamma, a, v)\}$ be one deterministic effect alternative, where γ is a condition, a an atom and $v \in \{0, 1\}$ its new value. For every pair (κ, α) the leaf contains one operator

$$\begin{aligned} \mathrm{pre}(o) &= \prec_j^*(t) \;\cup\; \big\{x_a = \kappa(a) : a \in \kappa \cap L_j\big\}, \\ \mathrm{eff}(o) &= \big\{x_a := v \text{ if } \gamma : (a, v) \in \alpha,\ a \in L_j\big\} \cup \{e_t := 1\}. \end{aligned} \tag{37}$$

A task that names an action is compiled through its action schemas; methods are used only when no action schema matches. The restriction to L_j in the condition and effect sets is intentional: constraints outside the leaf belong to a merge node. Effect atoms are still recorded in the leaf's `effect_atoms`, so that the merge can check which shared atoms a child plan establishes.

6.3 Compound tasks

A compound task t with no matching action schema gets one operator per grounded method instance $m\sigma$ and per DNF clause κ of $\phi_m\sigma$:

$$\mathrm{pre}(o) = \prec_j^*(t) \;\cup\; \big\{x_a = \kappa(a) : a \in \kappa \cap L_j\big\}, \qquad \mathrm{eff}(o) = \{e_t := 1\}. \tag{38}$$

If the domain declares no method whose task name and arity match t, a synthetic operator `task[`t`]` with no effect other than $e_t := 1$ is used. When primitive summaries are enabled (Section 8), a summarisable method is replaced by its macro operator; methods outside the summarisable fragment stay opaque.

6.4 Heuristic search

The leaf task is a finite multi-valued planning task. It is solved by greedy best-first search with the FF heuristic. The relaxed planning graph assigns each fact p a level

$$h(p) = \min_{o:\, p \in \mathrm{add}_o} \Big(\max_{q \in \mathrm{pre}_o} h(q) + \mathrm{cost}(o) \Big), \tag{39}$$

starting from $h(p) = 0$ for facts of the current state; axioms are applied to a fixed point on every layer. If a goal fact has infinite level the state is dead. Otherwise h_{FF} is the number of operators in the relaxed plan obtained by regressing from the goal facts through their cheapest achievers; the relaxed-plan operators that are applicable in the real state are returned as *preferred* actions. Search maintains two open lists per heuristic (preferred and non-preferred), uses a closed list, evaluates heuristics lazily when a node is expanded, and expands at most 10^5 nodes per leaf. Every operator has unit cost, so the leaf returns the first plan found, not a cost-optimal one.

7 Merging as a relaxed MINLP

7.1 The SGPlan penalty formulation

Let a merge node v have n_c children $C_1, \ldots, C_{n_c}$ with plans

$$\pi_i = \langle o_{i,1}, \ldots, o_{i,l_i} \rangle, \qquad \pi_i \models P_i, \tag{40}$$

and let $\mathcal{S}_v$ be the shared constraints attached to v. SGPlan's constraint-partitioning formulation is the mixed-integer nonlinear program

$$\begin{aligned} \min_{\pi_1,\ldots,\pi_{n_c},\, w} \quad & \sum_{i=1}^{n_c} \mathrm{cost}_i(\pi_i) + \sum_{c \in \mathcal{S}_v} w_c\, v_c(\pi_1, \ldots, \pi_{n_c}) & & \\ \text{s.t.} \quad & v_c(\pi_1, \ldots, \pi_{n_c}) = 0 & & \forall c \in \mathcal{S}_v, \\ & \pi_i \in \mathrm{Plans}(P_i) & & \forall i, \\ & w_c \geq 0 & & \forall c \in \mathcal{S}_v, \end{aligned} \tag{41}$$

where v_c is the violation of shared constraint c and w_c its penalty weight. PSPR does not re-plan inside the children: it fixes the π_i and optimizes only how they are interleaved.

7.2 Continuous relaxation

Let

$$L = \sum_{i=1}^{n_c} l_i, \qquad \ell_i = \frac{l_i}{L}, \qquad q_i \in [0, 1] \tag{42}$$

be the total number of steps, the normalized length of child i, and its continuous start position. Step k of child i (with $k = 0, \dots, l_i - 1$) occurs at the normalized time

$$\tau_{i,k} = q_i + \frac{k}{L}, \tag{43}$$

and the child completes at $q_i + \ell_i$. The makespan is replaced by a log-sum-exp smooth maximum

$$M_T(q) = T \log \sum_{i=1}^{n_c} \exp\left(\frac{q_i + \ell_i}{T}\right), \tag{44}$$

whose gradient with respect to q_i is the softmax weight

$$\frac{\partial M_T}{\partial q_i} = \frac{\exp\big((q_i + \ell_i)/T\big)}{\sum_j \exp\big((q_j + \ell_j)/T\big)}. \tag{45}$$

For a shared ordering constraint $a \prec b$, where a is step k of child i and b is step l of child j, define the signed margin

$$g_{ab}(q) = q_j + \frac{l}{L} - q_i - \frac{k}{L} - \frac{1}{L}. \tag{46}$$

It is non-negative exactly when a is scheduled strictly before b with at least one step of separation. Its exact violation is $[-g_{ab}]_+$, and its smooth violation is the one-sided hinge

$$v_{ab}(q) = T\,\mathrm{softplus}\left(-\frac{g_{ab}(q)}{T}\right) = T \log\left(1 + \exp\left(-\frac{g_{ab}(q)}{T}\right)\right), \tag{47}$$

with derivative

$$\frac{\partial v_{ab}}{\partial g_{ab}} = -\sigma\left(-\frac{g_{ab}}{T}\right), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}. \tag{48}$$

Hence the penalty stops pushing as soon as the constraint is comfortably satisfied.

Atom and equality constraints do not depend on the schedule. They enter the objective as constants, where an unsatisfied atom or equality contributes $w_c\,\phi(1)$ and a satisfied one contributes 0; they still drive feasibility and the weight adaptation.

7.3 The relaxed objective

With penalty function ϕ, make-span weight c, and weights w_c, the optimized objective is

$$J_T(q; w) = c\,M_T(q) + \sum_{c \in \mathcal{S}_v} w_c\,\phi\big(v_c(q)\big), \tag{49}$$

and its gradient is

$$\begin{aligned} \frac{\partial J_T}{\partial q_i} = c\,&\frac{\exp\big((q_i + \ell_i)/T\big)}{\sum_{j=1}^{n_c} \exp\big((q_j + \ell_j)/T\big)} \\ &+ \sum_{\substack{c=(a,b)\in\mathcal{S}_v \\ b \text{ in child } i}} w_c\,\phi'(v_c)\left(-\sigma\left(-\frac{g_c}{T}\right)\right) \\ &- \sum_{\substack{c=(a,b)\in\mathcal{S}_v \\ a \text{ in child } i}} w_c\,\phi'(v_c)\left(-\sigma\left(-\frac{g_c}{T}\right)\right). \end{aligned} \tag{50}$$

The three penalty spaces are

$$\phi_{\mathrm{lin}}(v) = v, \qquad \phi_{\mathrm{quad}}(v) = v^2 + v, \qquad \phi_{\exp}(v) = \frac{e^{4v} - 1}{4}, \tag{51}$$

with slopes 1, $2v + 1$ and e^{4v}.

7.4 Adam with box projection

The relaxed problem is minimized by Adam. With gradient g_t, moments

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{52}$$

bias corrections $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$, the update is

$$q^{(t+1)} = \Pi_{[0,1]}\left(q^{(t)} - \eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right), \tag{53}$$

where $\Pi_{[0,1]}$ is the coordinate-wise projection onto the unit box. The defaults are $\eta = 0.05$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, temperature $T = 0.05$, make-span weight $c = 1$, 10 outer rounds of 200 Adam steps each.

7.5 Adaptive weights and penalty continuation

After every outer round the exact violations v_c are measured. The weights are updated multiplicatively,

$$w_c \leftarrow \begin{cases} \mathrm{clamp}\big(\gamma \, w_c,\ w_{\min},\ w_{\max}\big) & \text{if } v_c > \tau, \\ \mathrm{clamp}\big(\delta \, w_c,\ w_{\min},\ w_{\max}\big) & \text{if } v_c \le \tau, \end{cases} \tag{54}$$

with growth $\gamma = 4$, decay $\delta = 1/2$, floor $w_{\min} = 10^{-3}$, ceiling $w_{\max} = 10^{9}$ and tolerance $\tau = 10^{-3}$. Decaying satisfied constraints keeps the active set well conditioned.

Let $V^{(r)}$ be the total exact violation after round r. A round is *stalled* when

$$V^{(r)} > (1-\rho)\, V^{(r-1)}, \qquad \rho = 0.05. \tag{55}$$

A stall advances the continuation from ϕ_{lin} to ϕ_{quad} and then to ϕ_{exp}, and resets Adam's moments so that the new curvature is measured from scratch. The best iterate of the whole run, ordered lexicographically by (total violation, objective), is retained, so a late divergent round cannot degrade the result.

If the merge node has no ordering constraints, if `iterations` is zero, or if the initial total violation is already at most τ, Adam is skipped and the initial (or learned) positions are decoded directly.

7.6 Decoding and exact validation

The relaxed positions are decoded by greedy list scheduling. Let $k_i \leftarrow 0$ for every child. Repeatedly select

$$i^* = \arg\min_{i:\, k_i < l_i} \left(q_i + \frac{k_i}{L} \right), \tag{56}$$

append step $o_{i^*,k_{i^*}}$ to the merged plan, and increment k_{i^*}. This interleaves the children while preserving the internal order of every child plan.

The decoded plan is then validated *exactly*, not through the smooth surrogates. For every shared ordering $a \prec b$, let $\mathrm{pos}(t)$ be the index of the first step of the decoded plan whose task index is t; the constraint holds iff both positions exist and $\mathrm{pos}(a) < \mathrm{pos}(b)$. A shared atom holds iff it belongs to

$$s_0 \ \cup \ \bigcup_{i=1}^{n_c} \big(\mathrm{final}_i \cup \mathrm{achieved}_i \big), \tag{57}$$

where final_i are the local atoms true at the end of child i and $\mathrm{achieved}_i$ are the ground effect atoms of its operators. An equality constraint is evaluated on its ground terms. If any constraint is still violated, the merge reports an error with the residual and weight of every offending constraint; there is no replanning at this stage.

Proposition 7.1 (Merge soundness)**.** *If a merge node returns successfully, then the decoded plan is a linearisation of the child plans that satisfies every shared constraint of that node, and its restriction to each child is exactly that child's plan.*

Proof. The decoder appends the next unscheduled step of one child at a time, so the projection onto each child is its original plan and its internal order is preserved. Feasibility is established by the exact validator above, which checks every shared ordering, atom and equality independently. A returned merge has passed that check for all constraints. □

Remark 7.2. *The continuous optimization is only a heuristic for* choosing *the interleaving. The smooth violation* (47) *is positive at the boundary, so feasibility is always decided by the exact test, never by the surrogate value.*

8 Hierarchical refinement

The abstract plan produced by the evaluation tree is a sequence of grounded top-level tasks, each annotated with the method chosen by the abstract operator. The refinement stage recursively replaces every compound task by a method instance and every primitive task by an action, producing a complete HTN plan.

8.1 The refinement relation

For a task t and state s, let $\mathcal{R}(t, s)$ be the finite set of pairs (π, s') such that π is a primitive sequence executable from s and s' is the resulting state. It is defined recursively:

- If t is a primitive action a and $s \models \mathrm{pre}_a$, then $\mathcal{R}(t, s) = \{(\langle a \rangle, \tau_a(s))\}$; otherwise $\mathcal{R}(t, s) = \varnothing$.
- If t is compound, let $m\sigma$ range over the grounded method instances with head t and $\phi_m\sigma$ true in s. For each such instance, refine the subtask network $TN_m\sigma$:
  - for a total order $t_1, \ldots, t_n$, form the sequential product

$$\mathcal{R}(t_1 \cdots t_n, s) = \Big\{(\pi_1 \cdots \pi_n, s_n) : (\pi_1, s_1) \in \mathcal{R}(t_1, s), \ldots, (\pi_n, s_n) \in \mathcal{R}(t_n, s_{n-1})\Big\};$$

  - for a partial order, enumerate the ready tasks (those whose predecessors are all scheduled), recurse on each ready task, and thread the state through the recursion. Equality and inequality constraints of the network are checked before expansion.

  The results are wrapped into a method step $\langle t, m\sigma, \cdot \rangle$.

8.2 Bounded search over the abstract plan

Let the abstract plan be $s_1, \ldots, s_m$ and let $\mathcal{C}_0 = \{(\varepsilon, s_0)\}$. For $j = 1, \ldots, m$,

$$\mathcal{C}_j = \mathrm{trunc}_{2048}\Big(\bigcup_{(\rho,s)\in\mathcal{C}_{j-1}} \{\rho \cdot \pi : (\pi, s') \in \mathcal{R}(s_j, s)\}\Big), \tag{58}$$

where $\rho \cdot \pi$ concatenates the prefix ρ with the new primitive sequence π. If $\mathcal{C}_j = \varnothing$ the refinement fails. The bound is $N_{\max} = 2048$ partial refinements per abstract step, the recursion depth is at most 32, and method grounding is bounded by B_g. A path-local visited set of (task, state) pairs prunes branches that rediscover the same subproblem; this is what stops recursive methods such as `get_to` through an intermediate location from cycling. The method recorded in the abstract plan is tried first, then the remaining methods in increasing order of subtask count.

After the last abstract step, the goal is evaluated on each candidate state. The first candidate whose goal is satisfied (or absent) is returned; if none satisfies the goal, the first candidate is

returned as a fallback and the caller can inspect `Refinement::goal`. The result is independently checked by `validate_refinement`, which re-executes method preconditions, subtask orderings, primitive effects, task coverage and the goal.

8.3 Primitive method summaries

A method $m = \langle c(\vec{x}), \phi_m, \langle a_1 \cdots a_n, \varnothing \rangle \rangle$ is summarised when every subtask a_i is primitive, the network is totally ordered, the method has no equality constraints, every action has exactly one DNF precondition clause κ_i and one unconditional effect alternative (A_i, D_i), and $A_i \cap D_i = \varnothing$.

The macro precondition is computed by backward regression. Set $R = \varnothing$, $A = \varnothing$, $D = \varnothing$ and process $i = n, n-1, \ldots, 1$:

1. for every literal $\ell = (p, b)$ in R: if $p \in A_i$, require $b = 1$ (otherwise fail); if $p \in D_i$, require $b = 0$ (otherwise fail); otherwise keep ℓ in R;
2. add the literals of κ_i to R, failing on a polarity conflict;
3. for every $p \in A_i$ with $p \notin D$, insert p into A;
4. for every $p \in D_i$ with $p \notin A$, insert p into D.

If no conflict occurred, the summary operator is

$$\mathrm{pre}(o) = \phi_m \ \wedge \bigwedge_{\ell \in R} \ell, \qquad \mathrm{add}(o) = A, \qquad \mathrm{del}(o) = D, \qquad \mathrm{add}(o) \mathrel{+}= \{\mathrm{exec}_t\}. \tag{59}$$

This is exact under the stated restrictions: the subtasks are totally ordered, deterministic and unconditional, so their composed effect is a single add/delete pair and their required precondition is the regressed literal set. Any method outside the fragment is left opaque.

9 Optional learned initialization

When the crate is compiled with the `neural` feature, the merger can be warm-started by a heterogeneous graph-attention network instead of the sequential start. The graph has one node per child plan, one node per shared constraint and one global node, with 32 features per node and a dense incidence adjacency including self-loops. Three normalized GAT layers compute

$$\alpha_{ij} = \mathrm{softmax}_j\Big(\mathrm{LeakyReLU}\big(a^\top [W h_i \,\|\, W h_j]\big)\Big), \qquad h_i' = \sigma\Big(\sum_{j \in N(i)} \alpha_{ij} W h_j\Big), \tag{60}$$

and two heads predict the normalized start positions q_i and the positive penalty weights w_c. Training targets are produced by running the deterministic merger from the sequential start and recording the optimized parameters.

The predictions are only an initialization. They are accepted when their shape matches the problem, all positions are finite and all weights are strictly positive; otherwise the merger falls back to the sequential start. The exact validator of Section 7 runs in all cases. The default build has no Burn dependency and uses the deterministic sequential start.

A second optional model (`PSPR_DECOMPOSITION_MODEL`) can rank method candidates and order ready subtasks during refinement. Its outputs are accepted only when they form a permutation of the candidate set; otherwise the baseline order is used. Both learned components are therefore guidance for search ordering, not sources of logical soundness.

10 Correctness, cost, and limitations

10.1 What is guaranteed

Proposition 10.1 (Leaf soundness)**.** *Every plan returned by a leaf search is executable in the leaf task and ends in a state where all markers e_t of the cluster are true. Each operator corresponds to a grounded action or method instance of the original domain.*

Proposition 10.2 (Conditional completeness of the tree)**.** *If every leaf is solved, every merge node succeeds, and the refinement stage returns a candidate whose goal is satisfied, then the returned primitive plan is a complete HTN plan of the initial task network and satisfies the state goal.*

Proof sketch. The merge proposition gives, by induction over the schedule, a plan of every subtree that satisfies all shared constraints attached to its nodes. The root therefore satisfies all constraints of TN_0. Refinement replaces each top-level task by a grounded method whose precondition holds in the state threaded through the recursion, and its subtasks are recursively refined into executable primitive actions. The goal check is performed on the final state, and `validate_refinement` repeats the check independently. □

10.2 Sources of incompleteness

The algorithm is deliberately satisficing and incomplete:

1. **Abstraction gap.** $\alpha(P)$ is not a relaxation of P (Section 2), so an abstract plan can be unrefinable and a valid HTN plan can have no abstract counterpart.
2. **Dropped shared atoms.** Leaves cannot test or establish shared atoms. The merge can only reorder existing child plans; it cannot replan a child that needs a shared atom as a precondition.
3. **Bounded enumeration.** Grounding, DNF expansion, leaf search, refinement depth and refinement breadth are all capped.
4. **No recursive decomposition in the abstract tree.** Compound tasks in a leaf are opaque method operators; decomposition happens only in the refinement stage.
5. **Deterministic fragment.** Probabilistic formulas are rejected, and disjunctive effects are compiled as an angelic choice over deterministic variants.

10.3 Cost

Let n_1 be the number of grounded tasks, n_2 the number of constraints, k the number of clusters, and n_c the largest number of children of a merge node.

- Grounding and incidence construction are linear in the number of grounded tasks and their schema atoms, up to the budget B_g and the DNF cap of 4096.
- Spectral clustering costs $O(n_1^3)$ per group for the eigendecomposition and $O(n_1 \cdot \mathrm{nnz}(A))$ for the Gram matrix. The number of k-median iterations is bounded by the implementation.
- The Gilpin ILP has $O(k^3)$ binary indicators and is NP-hard in general; it is solved by SCIP on the small cluster counts that occur in practice. The scheduling ILP has $O(k^2)$ binaries.
- A leaf search is bounded by 10^5 expansions and 5×10^6 generated states.
- A merge node costs $O(R \cdot S \cdot (n_c + |\mathcal{S}_v|))$ for R rounds of S Adam steps, plus $O(L)$ for decoding. There are $k - 1$ merge nodes, and every step of a child plan is decoded exactly once per node on the path to the root.

10.4 Implementation status

The pipeline described here is the one implemented by the current working tree. The library test suite passes (73 tests at the time of writing); the IPC integration tests are gated on an external HDDL benchmark checkout and the `hddl_analyzer` executable. The default build contains no neural code and no Burn dependency. The evaluator reports a plan as `solved` when every leaf is solved and the root merge succeeds; the state goal is reported separately and is only enforced by the hierarchical path. The default planner is therefore a fast solver of the task-network projection, while `--hierarchical` (stages C+G) is the complete-HTN mode.

Stage	Module
HDDL parsing and JSON decoding	`src/parsing/`
Domain and problem translation	`src/solver/domain.rs`, `problem.rs`
Grounding, incidence graph, formula compiler	`evaluation_tree/graph.rs`, `formula.rs`
Spectral clustering (BiSBM)	`partition/bisbm/`
Merge-level and scheduling ILPs	`partition/agglomeration/`
Tree construction and leaf evaluation	`evaluation_tree/clustering.rs`, `leaf.rs`
Leaf search (FF, GBFS)	`fast_downward/`
Relaxed SGPlan merger	`evaluation_tree/merge/`
Hierarchical refinement (C+G)	`evaluation_tree/hierarchical.rs`
Independent validation	`evaluation_tree/verify.rs`
Optional learned warm start	`evaluation_tree/merge/neural.rs`

Table 1: Mapping from the stages of the algorithm to the implementation.

References

[1] Y. Hsu, B. Wah, R. Huang, and Y. Chen. Constraint partitioning in penalty formulations for solving temporal planning problems. *Artificial Intelligence*, 170(3):187–211, 2006.

[2] S. Gilpin, T. Sandholm, and J. Sørensen. Formalizing hierarchical clustering as integer linear programming. In *Proceedings of AAAI*, 2013.

[3] F. Braun. Strong consistency guarantees for clustering high-dimensional bipartite graphs with the spectral method. arXiv:2304.06903, 2023.

[4] M. Helmert. The Fast Downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.

[5] J. Hoffmann and B. Nebel. The FF planning system: fast plan generation through heuristic search. *Journal of Artificial Intelligence Research*, 14:253–302, 2001.

[6] D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. In *Proceedings of ICLR*, 2015.